



FinBoard

Platforma do Symulacji i Analizy Portfela Inwestycyjnego

Dokumentacja Techniczna i Projektowa

Zespół Developerski:

Dariusz Szymanek	nr indeksu: 123456
Damian Zwolak	nr indeksu: 131542
Jakub Strzepak	nr indeksu: 131516
Łukasz Babiś	nr indeksu: 131401

26 stycznia 2026

Spis treści

1	Wstęp	3
1.1	Cel projektu	3
1.2	Zakres pracy	3
1.3	Uzasadnienie tematu	3
1.4	Struktura dokumentacji	3
2	Tło teoretyczne i wykorzystane technologie	5
2.1	Wprowadzenie	5
2.2	Finnhub API	5
2.3	Frontend	5
2.3.1	React i TypeScript	5
2.3.2	Vite	5
2.3.3	TailwindCSS	5
2.3.4	Zarządzanie Stanem i Dane	5
2.4	Backend	6
2.4.1	Python i FastAPI	6
2.4.2	Pydantic	6
2.4.3	SQLAlchemy	6
2.4.4	Biblioteki analityczne	6
2.5	Baza Danych	6
2.6	Infrastruktura i Narzędzia	6
2.6.1	Docker	6
2.6.2	System kontroli wersji	6
3	Analiza i wymagania	7
3.1	Wymagania Funkcjonalne	7
3.1.1	Zarządzanie Użytkownikami (Auth)	7
3.1.2	Dane Rynkowe (Market Data)	7
3.1.3	Zarządzanie Portfelem (Portfolio)	7
3.1.4	Alerty i Powiadomienia	7
3.2	Wymagania Niefunkcjonalne	8
3.2.1	Bezpieczeństwo	8
3.2.2	Wydajność i Skalowalność	8
3.2.3	Użyteczność (Usability)	8
4	Projekt systemu i architektura	9
4.1	Podział na Serwisy (Bounded Contexts)	9
4.1.1	Auth Service (Tożsamość i Dostęp)	9
4.1.2	Refdata Service (Dane Referencyjne)	9
4.1.3	Marketdata Service (Dane Rynkowe)	9
4.1.4	Portfolio Service (Portfel i Zlecenia)	10
4.1.5	Alerts Service (Powiadomienia Cenowe)	10
4.1.6	Backtest Service (Symulacje)	10
4.1.7	Pozostałe Serwisy Pomocnicze	10
4.2	Architektura Logiczna	10
4.3	Projekt Bazy Danych	11
4.3.1	Schemat Auth (Uwierzytelnianie)	11
4.3.2	Schemat Refdata (Dane Referencyjne)	12
4.3.3	Schemat Marketdata (Dane Rynkowe)	12

4.3.4	Schemat Portfolio (Portfel Użytkownika)	12
4.3.5	Schemat Alerts (Powiadomienia)	12
4.3.6	Schemat Backtest (Symulacje)	12
4.4	Struktura API	12
4.5	Integracje z zewnętrznymi API	13
4.5.1	Finnhub API - Dane czasu rzeczywistego	14
4.5.2	Yahoo Finance (yfinance) - Dane historyczne	14
4.5.3	Logika pobierania danych	14
5	Metodyka i organizacja pracy	15
5.1	Model zarządzania projektem	15
5.2	Hierarchia i struktura wymagań	15
5.2.1	Atrybuty zadań (Issues)	16
5.3	Proces wytwarzania oprogramowania (Workflow)	16
5.3.1	Branching Strategy i Code Review	16
5.3.2	Continuous Integration (CI)	16
6	Implementacja systemu	17
6.1	Struktura Projektu	17
6.2	Algorytmy biznesowe	17
6.2.1	Algorytm Ewaluacji Alertów	17
6.2.2	Algorytm Realizacji Zlecenia (Execution)	17
6.3	Backend (Python)	18
6.3.1	Główny Punkt Wejścia	18
6.3.2	Moduły Usługowe (Services)	18
6.3.3	Evaluator Alertów	18
6.4	Frontend (React)	18
6.4.1	Routing i Strony	18
6.4.2	Komponenty UI	18
6.4.3	Zarządzanie Stanem	18
6.4.4	Wykresy Finansowe	18
6.5	Bezpieczeństwo i Integracja	19
6.6	Interfejs użytkownika i responsywność	19
6.6.1	Techniczna realizacja RWD	19
6.6.2	Landing Page	19
6.6.3	Pulpit Inwestora (Dashboard)	19
6.6.4	Wykresy	20
7	Testowanie i weryfikacja	22
7.1	Instrukcja instalacji i uruchomienia	22
7.1.1	Wymagania wstępne	22
7.1.2	Uruchomienie Bazy Danych (Docker)	22
7.1.3	Inicjalizacja danych (Seeding)	22
7.1.4	Uruchomienie Backendu	22
7.1.5	Uruchomienie Frontendu	23
7.2	Wstęp do testowania	23
7.3	Testy Manualne (Scenariusze)	23
7.3.1	Scenariusz 1: Rejestracja i Pierwsze Logowanie	23
7.3.2	Scenariusz 2: Zakup Akcji	23
7.3.3	Scenariusz 3: Weryfikacja Alertów	23
7.4	Testy Automatyczne	24
7.4.1	Backend (Pytest)	24
7.4.2	Frontend (Vitest)	24
7.5	Wnioski z testów	24
8	Podsumowanie	25
8.1	Wnioski końcowe	25
8.2	Możliwości rozwoju	25

Rozdział 1

Wstęp

1.1 Cel projektu

Celem niniejszej pracy inżynierskiej jest zaprojektowanie i zaimplementowanie zaawansowanej aplikacji webowej o nazwie **FinBoard**. System ten ma służyć jako kompleksowa platforma do zarządzania wirtualnym portfelem inwestycyjnym, analizy rynków finansowych oraz śledzenia wyników strategii inwestycyjnych w czasie rzeczywistym.

Głównym założeniem projektu jest dostarczenie użytkownikowi narzędzia, które odwzorowuje funkcjonalności profesjonalnych platform tradingowych, jednocześnie zachowując przejrzystość i łatwość obsługi. Aplikacja umożliwia symulację handlu akcjami, śledzenie historii transakcji, analizę techniczną za pomocą interaktywnych wykresów oraz otrzymywanie powiadomień o zmianach cen (alerty).

1.2 Zakres pracy

Zakres prac obejmował realizację kompletnego systemu informatycznego składającego się z dwóch głównych komponentów:

- **Backendu:** Opartego na architekturze mikroservisowej (logicznej), zaimplementowanego w języku Python z wykorzystaniem frameworka FastAPI. Odpowiada on za logikę biznesową, autoryzację, obsługę bazy danych oraz integrację z zewnętrznymi dostawcami danych rynkowych.
- **Frontendu:** Zbudowanego jako aplikacja typu SPA (Single Page Application) w technologii React i TypeScript. Interfejs użytkownika został zaprojektowany z naciskiem na responsywność i nowoczesny design (TailwindCSS).

Dodatkowo, projekt obejmuje konfigurację środowiska uruchomieniowego przy użyciu kontenerów Docker, co zapewnia łatwą replikację środowiska i deployment.

1.3 Uzasadnienie tematu

W dobie rosnącej inflacji i niskich stóp procentowych, zainteresowanie rynkami kapitałowymi wśród inwestorów indywidualnych znacząco wzrosło. Jednakże, bariera wejścia w świat tradingu jest często wysoka, zarówno ze względu na ryzyko finansowe, jak i skomplikowane interfejsy istniejących narzędzi.

FinBoard wypełnia tę lukę, oferując bezpieczne środowisko "Paper Trading"(symulacji handlu), gdzie użytkownicy mogą testować swoje strategie bez ryzyka utraty realnych środków. Projekt łączy w sobie aspekty edukacyjne z zaawansowaną technologią webową, stanowiąc doskonały przykład nowoczesnej inżynierii oprogramowania w domenie FinTech.

1.4 Struktura dokumentacji

Niniejsza dokumentacja została podzielona na następujące rozdziały:

- **Rozdział 2** przedstawia tło teoretyczne i opisuje wykorzystane technologie.
- **Rozdział 3** zawiera analizę wymagań funkcjonalnych i нефункциональных.
- **Rozdział 4** opisuje architekturę systemu oraz projekt bazy danych.
- **Rozdział 5** szczegółowo omawia implementację poszczególnych modułów.
- **Rozdział 6** prezentuje proces testowania i weryfikacji systemu.

- **Rozdział 7** stanowi podsumowanie projektu i wnioski końcowe.

Rozdział 2

Tło teoretyczne i wykorzystane technologie

2.1 Wprowadzenie

Wybór technologii dla nowoczesnej aplikacji finansowej jest kluczowy dla zapewnienia jej wydajności, bezpieczeństwa oraz łatwości w utrzymaniu. W projekcie FinBoard zdecydowano się na podział aplikacji na dwie niezależne warstwy: interfejs użytkownika (Frontend) oraz serwer logiki biznesowej (Backend), komunikujące się ze sobą za pomocą interfejsu REST API.

2.2 Finnhub API

Finnhub to nowoczesna platforma typu "Market Data Provider", dostarczająca dane rynkowe w czasie rzeczywistym poprzez interfejs REST API. W systemie FinBoard pełni ona kluczową rolę w module ewaluacji alertów, zapewniając dostęp do aktualnych wycen instrumentów finansowych z giełd amerykańskich (US Stocks).

2.3 Frontend

Warstwa wizualna aplikacji została zaimplementowana w oparciu o ekosystem języka JavaScript/TypeScript.

2.3.1 React i TypeScript

Podstawą frontendu jest biblioteka **React**. Wybór ten podyktowany był jej komponentową architekturą, co pozwala na łatwe budowanie skomplikowanych interfejsów z mniejszych, reużywalnych bloków kodu. Jako język programowania wybrano **TypeScript**, który jest nadzbiorem JavaScript wprowadzającym statyczne typowanie. Zastosowanie TypeScriptu znacząco podnosi jakość kodu, ułatwia refaktoryzację oraz zapobiega wielu błędom już na etapie kompilacji.

2.3.2 Vite

Jako narzędzie do budowania i uruchamiania środowiska deweloperskiego wykorzystano **Vite**. Jest to nowoczesny bundler, który zapewnia błyskawiczne przeładowywanie modułów (HMR - Hot Module Replacement) oraz zoptymalizowane budowanie wersji produkcyjnej.

2.3.3 TailwindCSS

Do stylowania aplikacji użyto frameworka **TailwindCSS**. Jest to podejście "utility-first", gdzie zamiast pisać tradycyjne arkusze stylów CSS, stosuje się predefiniowane klasy bezpośrednio w kodzie HTML/JSX. Pozwala to na szybkie prototypowanie oraz spójność wizualną całego projektu (Design System).

2.3.4 Zarządzanie Stanem i Dane

Do zarządzania lokalnym stanem aplikacji wykorzystano React Context API. Komunikacja z API backendowym odbywa się przy użyciu nowoczesnych metod asynchronicznych (Fetch API), a prezentacja danych (wykresy finansowe) jest realizowana za pomocą bibliotek wyspecjalizowanych w wizualizacji danych.

2.4 Backend

Warstwa serwerowa odpowiada za logikę biznesową, przetwarzanie zleceń oraz bezpieczeństwo danych.

2.4.1 Python i FastAPI

Język **Python** został wybrany ze względu na jego dominującą pozycję w świecie analizy danych i finansów. Jako framework webowy zastosowano **FastAPI**. Jest to nowoczesne, wysoce wydajne (dzięki użyciu serwera ASGI - Uvicorn) narzędzie do budowania API. Kluczową zaletą FastAPI jest automatyczna walidacja danych wejściowych oraz generowanie dokumentacji OpenAPI (Swagger UI) w czasie rzeczywistym.

2.4.2 Pydantic

Do walidacji danych i zarządzania konfiguracją użyto biblioteki **Pydantic**. Gwarantuje ona poprawność typów danych przesyłanych między frontendem a backendem, co jest krytyczne w aplikacjach finansowych.

2.4.3 SQLAlchemy

Jako warstwę dostępu do danych (ORM - Object Relational Mapper) wykorzystano **SQLAlchemy**. Pozwala ona na operowanie na bazie danych za pomocą obiektów Pythona, jednocześnie dając możliwość wykonywania surowych zapytań SQL tam, gdzie wymagana jest najwyższa wydajność.

2.4.4 Biblioteki analityczne

Backend korzysta również z **Pandas** do manipulacji szeregami czasowymi (dane giełdowe OHLCV) oraz **yfinance** do pobierania historycznych danych rynkowych.

2.5 Baza Danych

Jako system zarządzania bazą danych (RDBMS) wybrano **PostgreSQL**. PostgreSQL oferuje zaawansowane funkcje niezbędne w systemach finansowych, takie jak:

- Transakcyjność (ACID).
- Wsparcie dla typów JSONB (przechowywanie nieschematyzowanych metadanych).
- Wydajne indeksowanie.
- Partycjonowanie tabel (przydatne przy dużych wolumenach danych historycznych).

2.6 Infrastruktura i Narzędzia

2.6.1 Docker

Cały system został skonteneryzowany przy użyciu platformy **Docker**. Definicja usług (frontend, backend, baza danych) znajduje się w pliku `docker-compose.yml`. Umożliwia to uruchomienie całego środowiska jedną komendą, niezależnie od systemu operacyjnego hosta.

2.6.2 System kontroli wersji

Projekt jest zarządzany przy użyciu systemu **Git**, co zapewnia śledzenie historii zmian oraz możliwość pracy zespołowej.

Rozdział 3

Analiza i wymagania

3.1 Wymagania Funkcjonalne

Na podstawie analizy potrzeb potencjalnych użytkowników oraz przeglądu istniejących rozwiązań rynkowych, zdefiniowano następujące wymagania funkcjonalne dla systemu FinBoard.

3.1.1 Zarządzanie Użytkownikami (Auth)

System musi umożliwiać identyfikację i autoryzację użytkowników.

1. **Rejestracja:** Użytkownik może utworzyć nowe konto podając imię, nazwisko, adres e-mail oraz hasło.
2. **Logowanie:** Dostęp do systemu jest możliwy po podaniu poprawnych danych uwierzytelniających. System wykorzystuje tokeny JWT (JSON Web Tokens) do utrzymania sesji.
3. **Profil Użytkownika:** Użytkownik ma wgląd w swoje dane osobowe oraz informacje o koncie (np. waluta bazowa).

3.1.2 Dane Rynkowe (Market Data)

System musi dostarczać aktualne (lub symulowane w czasie rzeczywistym) informacje o instrumentach finansowych.

1. **Przegląd Rynku:** Lista dostępnych instrumentów (akcje, kryptowaluty, ETF) z ich aktualnymi cenami.
2. **Wykresy:** Interaktywne wykresy świecowe (Candlestick) prezentujące historyczne ceny otwarcia, najwyższe, najniższe i zamknięcia (OHLCV) w różnych interwałach czasowych.
3. **Szczegóły Instrumentu:** Wyświetlanie szczegółowych metadanych, takich jak ticker, nazwa pełna, waluta kwotowana.

3.1.3 Zarządzanie Portfelem (Portfolio)

Kluczowy moduł pozwalający na symulację inwestycji.

1. **Składanie Zleceń:** Użytkownik może składać zlecenia kupna (BUY) i sprzedaży (SELL) instrumentów po cenie rynkowej (Market Order) lub z limitem ceny (Limit Order).
2. **Pozycje Otwarte:** Podgląd aktualnie posiadanych aktywów wraz z wyliczeniem zysku/straty (PnL - Profit and Loss) niezrealizowanego.
3. **Historia Transakcji:** Pełny rejestr zrealizowanych zleceń (Executions) oraz historii operacji na koncie (Wpłaty/Wypłaty).
4. **Rachunek Wirtualny:** System obsługuje wirtualne saldo gotówkowe, które zmienia się w wyniku transakcji.

3.1.4 Alerty i Powiadomienia

System wspiera aktywne monitorowanie rynku przez użytkownika.

1. **Definiowanie Alertów:** Możliwość ustawienia powiadomienia, gdy cena instrumentu przekroczy określony poziom (górny lub dolny).
2. **Lista Alertów:** Podgląd aktywnych i historycznych alertów w panelu użytkownika.
3. **Weryfikacja Alertów:** Mechanizm backendowy cyklicznie sprawdzający warunki alertów.

3.2 Wymagania Niefunkcjonalne

Oprócz funkcji biznesowych, system musi spełniać określone standardy jakościowe.

3.2.1 Bezpieczeństwo

- Hasła użytkowników muszą być przechowywane w formie zahasowanej (użycie algorytmu bcrypt).
- Komunikacja z API musi być zabezpieczona (HTTPS w produkcji, walidacja tokenów w nagłówkach).
- Ochrona przed atakami typu SQL Injection (zastosowanie parametrów w zapytaniach / ORM).

3.2.2 Wydajność i Skalowalność

- Czas odpowiedzi API nie powinien przekraczać 200 ms dla standardowych operacji.
- Architektura bazy danych (indeksy) powinna zapewniać szybki dostęp do danych historycznych.
- Możliwość łatwego uruchomienia wielu instancji serwisu (konteneryzacja).

3.2.3 Użyteczność (Usability)

- Interfejs musi być responsywny (RWD), dostosowując się do ekranów komputerów i urządzeń mobilnych.
- Projekt graficzny powinien być nowoczesny i czytelny (tryb ciemny/jasny, wyraźne kontrasty).

Rozdział 4

Projekt systemu i architektura

4.1 Podział na Serwisy (Bounded Contexts)

W ramach przyjętej architektury, logika biznesowa systemu została podzielona na autonomiczne domeny (serwisy). Taki podział zapewnia separację odpowiedzialności (Separation of Concerns) i ułatwia przyszłe skalowanie poszczególnych modułów. Poniżej przedstawiono szczegółową charakterystykę poszczególnych serwisów.

4.1.1 Auth Service (Tożsamość i Dostęp)

Serwis odpowiedzialny za bezpieczeństwo i zarządzanie sesjami użytkowników. Pełni rolę strażnika dostępu do pozostałych zasobów systemu.

- **Zakres:** Rejestracja, logowanie, obsługa uwierzytelniania dwuskładnikowego (2FA), odświeżanie tokenów (refresh tokens), reset hasła oraz audyt logowań.
- **Kluczowa rola:** Identyfikator `users.id` generowany przez ten serwis jest globalnym unikalnym identyfikatorem użytkownika wykorzystywanym we wszystkich innych serwisach.
- **Wykorzystywane dane:** `auth.users`, `auth.user_sessions`.

4.1.2 Refdata Service (Dane Referencyjne)

Stanowi "źródło prawdy" dla definicji instrumentów finansowych. Inne serwisy nie przechowują pełnych danych o instrumencie, lecz jedynie referencje do identyfikatorów zarządzanych przez ten moduł.

- **Zakres:** Zarządzanie słownikami instrumentów, giełd, rynków, kalendarzy sesyjnych oraz kursami walut (FX).
- **Kluczowa rola:** Centralizacja metadanych, co zapobiega redundancji informacji o tickerach czy godzinach handlu w innych modułach.
- **Wykorzystywane dane:** `refdata.instruments`, `refdata.venues`, `refdata.listings`, `refdata.calendars`.

4.1.3 Marketdata Service (Dane Rynkowe)

Moduł odpowiedzialny za pozyskiwanie, przetwarzanie (ETL) i udostępnianie danych rynkowych.

- **Zakres:** Pobieranie świec OHLCV (historycznych), snapshotów cenowych (Quotes) oraz opcjonalnie newsów i sentymentu.
- **Zależności:**
 - Udostępnia historyczne dane OHLCV dla **Backtest Service**.
 - Udostępnia bieżące ceny (Quotes) dla **Alerts Service** (do sprawdzania warunków) oraz **Portfolio Service** (do bieżącej wyceny aktywów).
- **Wykorzystywane dane:** `marketdata.ohlcv`, `marketdata.quotes_latest`.

4.1.4 Portfolio Service (Portfel i Zlecenia)

Serce logiki transakcyjnej systemu. Działa w oparciu o model księgi głównej (ledger), zapewniając spójność operacji finansowych.

- **Zakres:** Obsługa kont "papierowych"(demo), zarządzanie pozycjami, przyjmowanie zleceń i ich wirtualna egzekucja.
- **Logika wyceny:** Serwis ten nie aktualizuje cen samodzielnie. W celu wyceny wartości portfela (PnL) oraz otwartych pozycji, odpytuje **Marketdata Service** o aktualne kursy.
- **Separacja:** Wyraźnie rozdzielono pozycje (stan aktualny) od egzekucji (historia transakcji).
- **Wykorzystywane dane:** `portfolio.accounts`, `portfolio.orders`, `portfolio.positions`, `portfolio.executions`.

4.1.5 Alerts Service (Powiadomienia Cenowe)

Silnik regułowy sprawdzający warunki rynkowe zdefiniowane przez użytkownika.

- **Zakres:** Obsługa alertów cenowych, procentowych, wolumenowych oraz prostych wskaźników (SMA).
- **Działanie:** Cyklicznie weryfikuje warunki w oparciu o bieżące ceny z Marketdata Service (nie korzysta z danych historycznych). Po spełnieniu warunku generuje zdarzenie `AlertTriggered`, które jest przekazywane do Notify Service.
- **Wykorzystywane dane:** `alerts.alerts`.

4.1.6 Backtest Service (Symulacje)

Moduł analityczny służący do weryfikacji strategii inwestycyjnych na danych historycznych.

- **Zakres:** Definiowanie strategii, uruchamianie testów, generowanie krzywej kapitału (Equity Curve) oraz obliczanie metryk skuteczności (np. Sharpe Ratio).
- **Izolacja:** Działa wyłącznie na danych historycznych OHLCV, nie ma dostępu do bieżącego rynku (Live).
- **Wykorzystywane dane:** `backtest.strategies`, `backtest.backtests`, `backtest.backtest_metrics`.

4.1.7 Pozostałe Serwisy Pomocnicze

- **Watchlist Service:** Przechowuje listy obserwowanych instrumentów. Dbą o unikalność par (`user_id`, `instrument_id`) w ramach listy.
- **Notify Service:** Infrastrukturalny serwis odpowiedzialny za wysyłkę powiadomień (Email, WebPush) w reakcji na zdarzenia systemowe, np. wyzwolenie alertu cenowego.

4.2 Architektura Logiczna

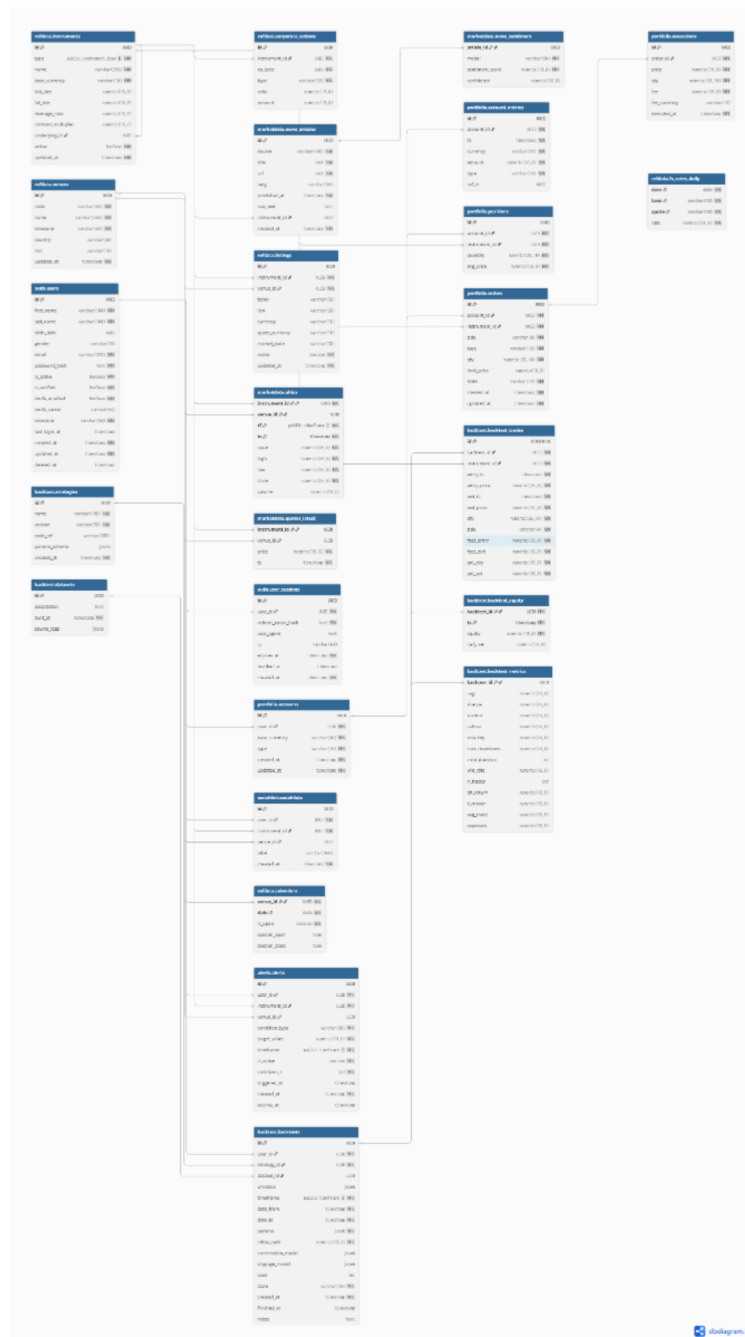
System FinBoard został zaprojektowany zgodnie z wzorcem architektury wielowarstwowej (N-tier) oraz zasadami architektury mikroservisowej (na poziomie logicznym, wdrożeniowo jest to modułarny monolit w kontenerach).

Główne komponenty systemu to:

1. **Klient (Frontend):** Aplikacja przeglądarkowa (SPA) odpowiedzialna za interakcję z użytkownikiem i prezentację danych.
2. **API Gateway / Serwer Aplikacji (Backend):** Centralny punkt dostępu do usług biznesowych. Przyjmuje żądania HTTP, przetwarza je i zwraca odpowiedzi w formacie JSON.
3. **Baza Danych (Data Tier):** Relacyjna baza danych przechowująca trwały stan systemu.
4. **Zewnętrzne API (Market Data Provider):** Źródło danych rynkowych (np. Yahoo Finance, Alpha Vantage) integracja realizowana przez backend.
5. **Workers (Background Tasks):** Procesy działające w tle, np. `run_alert_evaluator.py`, odpowiedzialne za sprawdzanie warunków alertów.

4.3 Projekt Bazy Danych

Baza danych PostgreSQL została podzielona na logiczne schematy (Schemas), co odzwierciedla modułową budowę systemu. Poniżej przedstawiono opis kluczowych tabel w każdym ze schematów.



Rysunek 4.1: Schemat ERD Bazy Danych.

Poniżej szczegółowo opisano poszczególne moduły bazy danych:

4.3.1 Schemat Auth (Uwierzytelnianie)

Przechowuje dane użytkowników i sesji.

- **users:** Główna tabela użytkowników (ID, email, hasło, status). Klucz główny to UUID.
- **user_sessions:** Tabela przechowująca aktywne sesje i tokeny odświeżające (Refresh Tokens), co pozwala na zarządzanie wylogowywaniem i bezpieczeństwem.

4.3.2 Schemat Refdata (Dane Referencyjne)

Słowniki i definicje instrumentów.

- **instruments:** Definicje instrumentów finansowych (typ, waluta bazowa).
- **venues:** Giełdy i rynki (np. NASDAQ, GPW).
- **listings:** Konkretnie notowania instrumentów na danej giełdzie (Ticker, ISIN).
- **calendars:** Kalendarz sesji giełdowych (dni i godziny handlu).

4.3.3 Schemat Marketdata (Dane Rynkowe)

Dane historyczne i bieżące o cenach.

- **ohlcv:** Tabela przechowująca świece cenowe (Open, High, Low, Close, Volume) dla różnych interwałów czasowych (m1, h1, d1). Jest to największa tabela w systemie.
- **quotes_latest:** Najnowsza znana cena dla danego instrumentu (cache).

4.3.4 Schemat Portfolio (Portfel Użytkownika)

Logika inwestycyjna.

- **accounts:** Konta inwestycyjne użytkowników (np. konto Demo, konto Live).
- **account_entries:** Historie operacji gotówkowych (wpłaty, wypłaty, opłaty).
- **orders:** Zlecenia złożone przez użytkownika (oczekujące, zrealizowane, anulowane).
- **executions:** Szczegóły realizacji zleceń (konkretna cena, ilość, prowizja).
- **positions:** Aktualnie otwarte pozycje (ilość posiadanych aktywów, średnia cena zakupu). Tabela ta jest aktualizowana automatycznie po każdej transakcji.

4.3.5 Schemat Alerts (Powiadomienia)

Konfiguracja alertów cenowych.

- **price_alerts:** Definicje alertów ustawionych przez użytkownika (np. "Powiadom gdy cena AAPL > 150 USD"). Przechowuje status aktywności i datę ostatniego wyzwolenia.

4.3.6 Schemat Backtest (Symulacje)

Moduł do testowania strategii historycznych.

- **strategies:** Definicje strategii inwestycyjnych.
- **backtests:** Uruchomione symulacje (parametry, zakres dat).
- **backtest_trades:** Transakcje wygenerowane przez symulację.
- **backtest_metrics:** Wyniki skuteczności strategii (CAGR, Sharpe Ratio, Drawdown).

4.4 Struktura API

Interfejs programistyczny (API) został zaprojektowany zgodnie ze standardem RESTful. Zasoby są reprezentowane przez adresy URL, a operacje na nich wykonywane są za pomocą metod HTTP:

default		^
GET	/health Health Check	▼
auth		^
POST	/auth/register Register User	▼
POST	/auth/login Login	▼
GET	/auth/me Read Me	🔒 ▼
refdata		^
GET	/refdata/listings Search Listings	▼
GET	/refdata/listings/{listing_id} Get Listing Detail	▼
watchlist		^
GET	/watchlist/me Get My Watchlist	🔒 ▼
POST	/watchlist Add To Watchlist	🔒 ▼
DELETE	/watchlist/{watchlist_id} Remove From Watchlist	🔒 ▼
marketdata		^
GET	/marketdata/quote Get Quote	▼
GET	/marketdata/ohlcv Get Ohlcv	▼
POST	/marketdata/quotes Get Quotes Batch	▼
portfolio		^
GET	/portfolio/health Portfolio Health	▼
GET	/portfolio/accounts/me Get My Accounts	🔒 ▼
POST	/portfolio/accounts Create Paper Account	🔒 ▼
POST	/portfolio/accounts/{account_id}/deposit Deposit Cash	🔒 ▼
GET	/portfolio/accounts/{account_id}/balance Get Balance	🔒 ▼
POST	/portfolio/orders Place Market Order	🔒 ▼
GET	/portfolio/orders/me List My Orders	🔒 ▼
POST	/portfolio/orders/{order_id}/fill Fill Order	🔒 ▼
GET	/portfolio/positions/me Get My Positions	🔒 ▼
GET	/portfolio/executions/me Get My Executions	🔒 ▼
alerts		^
POST	/alerts Create Alert	🔒 ▼
GET	/alerts/me List My Alerts	🔒 ▼
PATCH	/alerts/{alert_id} Update Alert	🔒 ▼
DELETE	/alerts/{alert_id} Delete Alert	🔒 ▼

Rysunek 4.2: Finboard API - Swagger UI.

Ścieżki są zgrupowane w routery odpowiadające poszczególnym modułom biznesowym.

4.5 Integracje z zewnętrznymi API

System FinBoard działa jako agregator danych rynkowych, wykorzystując hybrydowe podejście do pozyskiwania informacji z zewnętrznych źródeł (SaaS). Pozwala to na uniknięcie kosztownego utrzymywania własnych serwerów danych giełdowych.

4.5.1 Finnhub API - Dane czasu rzeczywistego

Finnhub został wybrany jako główne źródło danych dla modułów wymagających wysokiej aktualności (Low Latency).

- **Zastosowanie:** Odświeżanie bieżących wycen w portfelu użytkownika oraz zasilanie silnika *Evaluator Alertów*.
- **Charakterystyka:** Komunikacja odbywa się przez protokół REST z wykorzystaniem klucza API. Przykładowy format zapytania o cenę akcji: `/quote?symbol=AAPL`.

4.5.2 Yahoo Finance (yfinance) - Dane historyczne

Do zadań analitycznych, które nie wymagają odświeżania co sekundę, wykorzystywana jest biblioteka **yfinance**.

- **Zastosowanie:** Pobieranie szerokich zakresów danych historycznych (OHLCV) dla wykresów oraz dostarczanie danych wejściowych do modułu *Backtest Service*.
- **Zalety:** Brak restrykcyjnych limitów zapytań dla danych historycznych, co umożliwia generowanie wykresów o dużej gęstości bez dodatkowych opłat.

4.5.3 Logika pobierania danych

Backend systemu posiada warstwę abstrakcji, która decyduje, z którego API skorzystać w zależności od kontekstu wywołania:

```
1 def fetch_market_data(symbol: str, mode: str):  
2     if mode == "REALTIME":  
3         return finnhub_client.get_quote(symbol)  
4     else:  
5         return yf.download(symbol, period="1y")
```

Listing 4.1: Uproszczona logika wyboru źródła danych

Rozdział 5

Metodyka i organizacja pracy

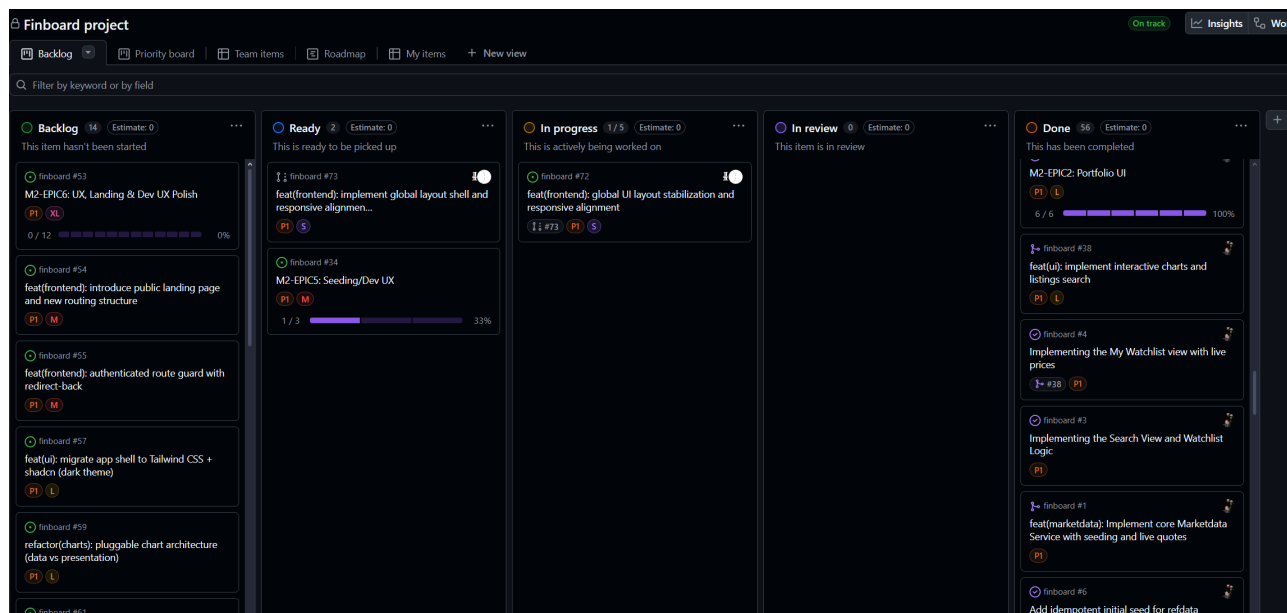
5.1 Model zarządzania projektem

Proces wytwórczy systemu FinBoard został oparty o zwinne metodyki zarządzania projektem (Agile). Zespół zdecydował się na podejście hybrydowe, łączące elastyczność metodyki **Kanban** z iteracyjnym charakterem metodyki **Scrum**.

Jako główne narzędzie do zarządzania zadaniami wykorzystano platformę **GitHub Projects**. Pozwoliło to na pełną integrację kodu źródłowego z systemem śledzenia postępów (Issue Tracking) oraz transparentność procesu dla wszystkich członków zespołu.

Tablica zadań została podzielona na klasyczne kolumny odzwierciedlające status prac:

- **Backlog:** Zadania zaplanowane do realizacji.
- **In Progress:** Zadania aktualnie wykonywane przez developerów.
- **In Review:** Zadania oczekujące na weryfikację (Code Review).
- **Done:** Zadania zakończone i przetestowane.



Rysunek 5.1: Widok tablicy projektowej w GitHub Projects.

5.2 Hierarchia i struktura wymagań

W celu zachowania porządku i mierzalności postępów, wdrożono trzystopniową hierarchię zadań:

1. **Milestones (Kamienie Milowe):** Projekt został podzielony na 3 główne etapy (Milestones), wyznaczające kluczowe terminy dostarczenia funkcjonalności MVP oraz kolejnych wersji systemu.

2. **Epics (Epiki):** W ramach kamieni milowych zdefiniowano większe obszary funkcjonalne (np. "Moduł Uwierzytelniania", "Silnik Backtestów").
3. **Issues (Zadania):** Najmniejsze jednostki pracy, realizowane przez pojedynczych programistów.

5.2.1 Atrybuty zadań (Issues)

Każde zadanie w systemie GitHub Projects posiadało ściśle zdefiniowany zestaw atrybutów, co usprawniło priorytetyzację i planowanie:

- **Priorytet:** Określał pilność zadania (np. *High, Medium, Low, Critical*).
- **Estymacja (Size):** Określała złożoność zadania (np. w skali koszulkowej XS, S, M, L, XL), co pozwalało na lepsze równoważenie obciążenia zespołu.
- **Acceptance Criteria (AC):** Każde zadanie musiało posiadać zdefiniowaną listę Kryteriów Akceptacji. Były to konkretne warunki, które musiały zostać spełnione, aby zadanie mogło zostać uznane za wykonane (Definition of Done).

5.3 Proces wytwarzania oprogramowania (Workflow)

W projekcie zastosowano rygorystyczne standardy zapewnienia jakości kodu, opierające się na procesie Continuous Integration (CI) oraz wzajemnej weryfikacji kodu.

5.3.1 Branching Strategy i Code Review

Praca z kodem odbywała się w modelu *Feature Branch Workflow*. Bezpośrednie zmiany w głównej gałęzi (`main`) były zablokowane.

1. Programista tworzył nową gałąź dla realizowanego zadania.
2. Po zakończeniu implementacji i spełnieniu AC, otwierany był **Pull Request (PR)**.
3. Każdy PR wymagał zatwierdzenia (Code Review) przez co najmniej jednego innego członka zespołu. Weryfikowano czytelność kodu, zgodność z architekturą oraz brak błędów logicznych.

5.3.2 Continuous Integration (CI)

Proces weryfikacji został zautomatyzowany przy użyciu potoków CI (np. GitHub Actions). Dla każdego Pull Requestu automatycznie uruchamiano:

- Testy jednostkowe i integracyjne.
- Budowanie projektu w celu wykrycia błędów kompilacji.

Zadanie mogło zostać scalone (Merge) z główną gałęzią tylko w przypadku, gdy wszystkie testy CI zakończyły się sukcesem (kolor zielony), a kod uzyskał aprobatę podczas Review.

Rozdział 6

Implementacja systemu

6.1 Struktura Projektu

Kod źródłowy został podzielony na dwa główne katalogi: backend oraz frontend, co ułatwia niezależny rozwój i wdrażanie obu części systemu.

6.2 Algorytmy biznesowe

W tej sekcji przedstawiono kluczowe algorytmy sterujące logiką biznesową systemu FinBoard. Procesy te są realizowane asynchronicznie przez dedykowane procesy tła (Workers).

6.2.1 Algorytm Ewaluacji Alertów

Ewaluator alertów działa w pętli, sprawdzając zdefiniowane przez użytkowników warunki rynkowe. Poniższy pseudokod i opis ilustrują logikę sprawdzania ceny.

Rysunek 6.1: Schemat blokowy procesu ewaluacji alertów.

Pseudokod algorytmu:

```
1 # Pobierz wszystkie aktywne alerty
2 alerts = db.query(Alert).filter(Alert.status == "ACTIVE").all()
3
4 for alert in alerts:
5     current_price = market_data.get_latest_price(alert.symbol)
6
7     if alert.condition == "ABOVE" and current_price >= alert.target_price:
8         trigger_alert(alert, current_price)
9     elif alert.condition == "BELOW" and current_price <= alert.target_price:
10        trigger_alert(alert, current_price)
```

Listing 6.1: Logika sprawdzania warunku alertu

6.2.2 Algorytm Realizacji Zlecenia (Execution)

Proces ten odpowiada za zmianę stanu portfela użytkownika po zakupie lub sprzedaży aktywów.

Kluczowe kroki procesu:

1. **Walidacja:** Sprawdzenie, czy użytkownik posiada wystarczające środki (przy zakupie) lub odpowiednią ilość jednostek aktywa (przy sprzedaży).
2. **Blokada środków:** Tymczasowe zabezpieczenie kapitału pod zlecenie.
3. **Aktualizacja Portfela:** Rekalkulacja średniej ceny zakupu (Break Even Price) według wzoru:

$$P_{avg} = \frac{(Q_{old} \cdot P_{old}) + (Q_{new} \cdot P_{new})}{Q_{old} + Q_{new}} \quad (6.1)$$

4. **Finalizacja:** Zmiana statusu zlecenia na EXECUTED i zapisanie transakcji w tabeli executions.

6.3 Backend (Python)

Aplikacja serwerowa znajduje się w katalogu `backend/src`. Zastosowano podział na moduły funkcjonalne ("services"), co pozwala na zachowanie czystości architektury.

6.3.1 Główny Punkt Wejścia

Plik `main.py` inicjalizuje aplikację FastAPI, konfiguruje oprogramowanie pośredniczące (Middleware) takie jak CORS (Cross-Origin Resource Sharing) oraz dołącza routery z poszczególnych modułów.

6.3.2 Moduły Usługowe (Services)

Każdy katalog wewnątrz `src` (np. `auth_service`, `portfolio_service`) zawiera:

- **router.py**: Definicje endpointów API.
- **service.py**: Logika biznesowa.
- **schemas.py**: Modele danych Pydantic (DTO - Data Transfer Objects).
- **models.py**: Modele bazodanowe SQLAlchemy (opcjonalnie, jeśli są specyficzne dla modułu).

6.3.3 Evaluator Alertów

W katalogu `scripts` zaimplementowano skrypt `run_alert_evaluator.py`. Jest to niezależny proces, który:

1. Łączy się z bazą danych.
2. Pobiera aktywne alerty użytkowników.
3. Sprawdza aktualne ceny rynkowe (korzystając z serwisu Market Data).
4. W przypadku spełnienia warunku (np. cena powyżej progu), oznacza alert jako "wyzwolony" i (w przyszłości) wysyła powiadomienie (email/push).

6.4 Frontend (React)

Aplikacja kliencka znajduje się w katalogu `frontend/src`.

6.4.1 Routing i Strony

Wykorzystano `React Router` do nawigacji po stronie klienta. Główne widoki aplikacji znajdują się w folderze `pages`:

- **LoginPage / RegisterPage**: Formularze uwierzytelniania.
- **ChartPage**: Zaawansowany widok z wykresem giełdowym.
- **PortfolioPage**: Pulpit inwestora z podsumowaniem konta, listą pozycji i formularzem zleceń.

6.4.2 Komponenty UI

W folderze `components` znajdują się reużywalne elementy interfejsu, takie jak przyciski, tabele, modale czy elementy formularzy. Pozwala to na zachowanie spójności wizualnej w całej aplikacji.

6.4.3 Zarządzanie Stanem

Aplikacja wykorzystuje `React Context` do przechowywania globalnych danych.

- **AuthContext**: Przechowuje informacje o zalogowanym użytkowniku i tokenie sesji.
- **AlertsContext**: Zarządza stanem powiadomień wyświetlanych użytkownikowi (np. komunikaty sukcesu/błędu).

6.4.4 Wykresy Finansowe

Do prezentacji danych giełdowych wykorzystano bibliotekę **Recharts**. Zaimplementowano dedykowane komponenty do rysowania wykresów świecowych, linii średnich kroczących (SMA) oraz wskaźników technicznych (np. RSI).

6.5 Bezpieczeństwo i Integracja

Zaimplementowano komponent `RequireAuth`, który chroni prywatne trasy aplikacji przed dostępem osób niezalogowanych. W przypadku wygaśnięcia sesji, użytkownik jest automatycznie przekierowywany na stronę logowania.

6.6 Interfejs użytkownika i responsywność

Aplikacja FinBoard została zaprojektowana zgodnie z zasadą *Mobile First*. Dzięki temu interfejs jest w pełni responsywny i czytelny na każdym urządzeniu.

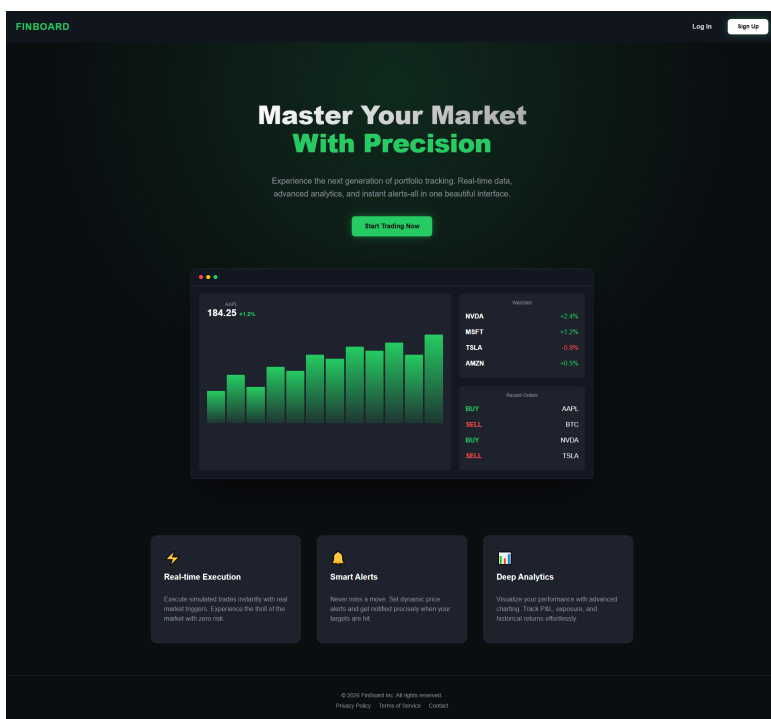
6.6.1 Techniczna realizacja RWD

Do budowy responsywnego interfejsu wykorzystano framework TailwindCSS:

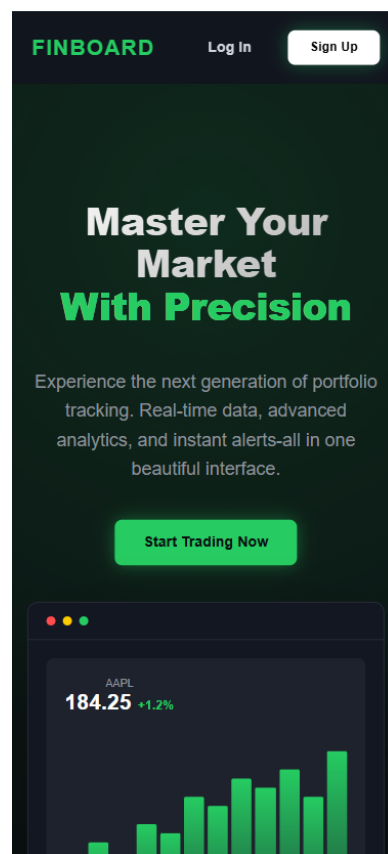
- **Grid System:** Zmiana liczby kolumn z 1 (mobile) na wiele (desktop).
- **Breakpoints:** Ukrywanie zbędnych elementów nawigacji na telefonach.
- **Flexbox:** Dynamiczne centrowanie kart produktów i wykresów.

6.6.2 Landing Page

Strona główna zachęca użytkownika do rejestracji i prezentuje główne zalety systemu.



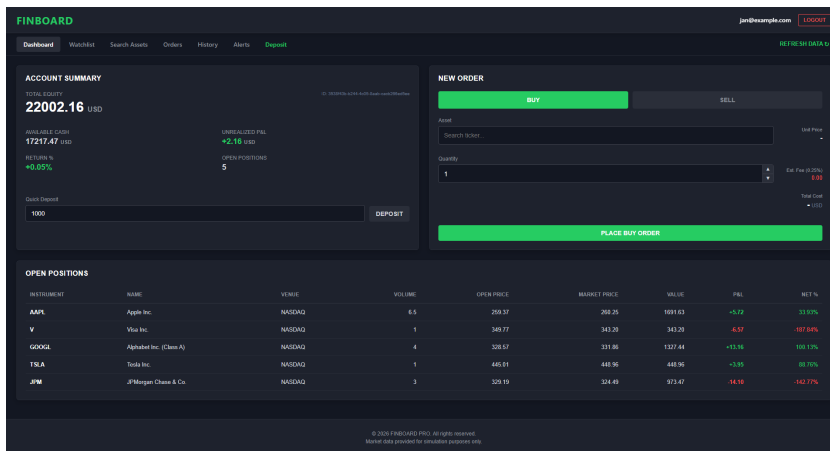
Rysunek 6.2: Strona główna w wersji desktopowej.



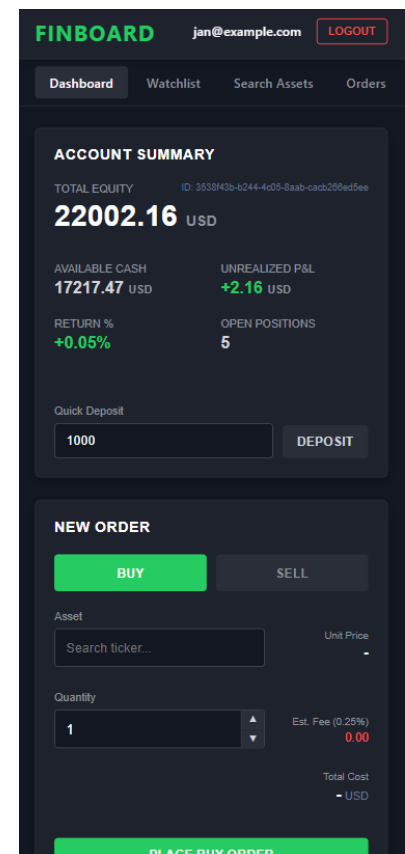
Rysunek 6.3: Strona główna w wersji mobilnej.

6.6.3 Pulpit Inwestora (Dashboard)

Panel główny pozwala na szybki podgląd stanu konta, aktualnych pozycji oraz składanie nowych zleceń kupna/sprzedaży.



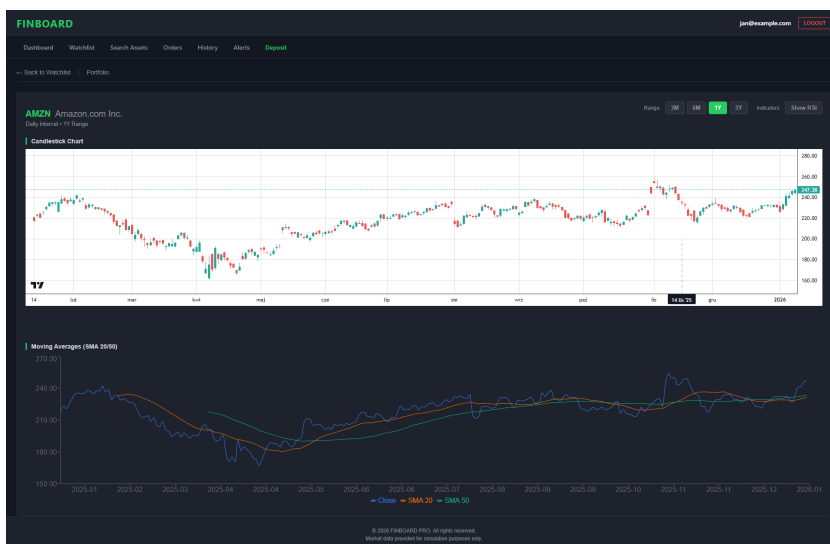
Rysunek 6.4: Pulpit inwestora (Desktop).



Rysunek 6.5: Pulpit (Mobile).

6.6.4 Wykresy

System oferuje zaawansowane wykresy świecowe oraz wskaźniki techniczne (SMA, RSI), które dostosowują swoją rozdzielczość do ekranu.



Rysunek 6.6: Widok wykresu (Desktop).



Rysunek 6.7: Widok wykresu (Mobile).

Rozdział 7

Testowanie i weryfikacja

7.1 Instrukcja instalacji i uruchomienia

FinBoard to panel analityczny rynków finansowych, umożliwiający śledzenie notowań giełdowych, analizę danych historycznych oraz wirtualne inwestowanie, zasilany przez darmowe API giełdowe.

7.1.1 Wymagania wstępne

Poniższa instrukcja zakłada, że w środowisku lokalnym zainstalowano następujące narzędzia: **Docker Desktop**, **Node.js** oraz **Python 3.10+**.

Konfiguracja środowiska

1. Sklonuj repozytorium na dysk lokalny.
2. W katalogu głównym projektu skopiuj plik `.env.example` i utwórz nowy plik o nazwie `.env`. Uzupełnij go wymaganymi kluczami API.

7.1.2 Uruchomienie Bazy Danych (Docker)

Upewnij się, że aplikacja **Docker Desktop** jest uruchomiona. Otwórz terminal w głównym katalogu projektu (`/finboard`) i wykonaj polecenia:

```
1 docker compose down
2 docker compose up -d
```

Listing 7.1: Uruchomienie kontenerów Docker

7.1.3 Inicjalizacja danych (Seeding)

Otwórz drugi terminal (PowerShell) w katalogu backendu (`/finboard/backend`).

Aktywacja środowiska wirtualnego

```
1 .\venv\Scripts\Activate
```

Następnie przejdź do katalogu głównego, wykonując `cd ...`

Uruchomienie skryptu zasilającego

```
1 .\seed_OHLCV.ps1
```

Listing 7.2: Skrypt seedujący dane OHLCV

7.1.4 Uruchomienie Backendu

Wróć do katalogu `/finboard/backend` w terminalu PowerShell i uruchom serwer deweloperski:

```
1 uvicorn src.main:app --reload
```

API będzie dostępne pod adresem: <http://localhost:8000>

7.1.5 Uruchomienie Frontendu

Otwórz trzeci terminal w katalogu /frontend.

Instalacja zależności i start aplikacji

```
1 npm install
2 npm run dev
```

Panel użytkownika będzie dostępny pod adresem: <http://localhost:5173>

Znane ograniczenia

- Historyczne dane OHLCV są pobierane i zapisywane ręcznie przy użyciu Yahoo Finance (skrypt seedujący).
- Notowania w czasie rzeczywistym są pobierane z serwisu Finnhub (darmowy plan).

7.2 Wstęp do testowania

Proces weryfikacji oprogramowania jest nieodłącznym elementem cyklu życia projektu informatycznego. W przypadku systemu FinBoard, testowanie odbywało się na wielu poziomach, obejmując zarówno weryfikację funkcjonalną interfejsu użytkownika, jak i testy jednostkowe logiki biznesowej backendu.

7.3 Testy Manualne (Scenariusze)

Przeprowadzono szereg testów manualnych, odtwarzających typowe ścieżki użytkownika (User Journeys).

7.3.1 Scenariusz 1: Rejestracja i Pierwsze Logowanie

- **Krok 1:** Użytkownik wchodzi na stronę rejestracji.
- **Krok 2:** Wypełnia formularz poprawnymi danymi.
- **Krok 3:** System tworzy konto i przekierowuje do logowania.
- **Krok 4:** Użytkownik loguje się.
- **Oczekiwany Rezultat:** Użytkownik widzi Dashboard. W bazie danych (tabela `auth.users`) pojawia się nowy rekord.

7.3.2 Scenariusz 2: Zakup Akcji

- **Krok 1:** Użytkownik wyszukuje spółkę (np. Apple) i przechodzi do jej wykresu.
- **Krok 2:** Wybiera typ zlecenia "Market" i wpisuje ilość (np. 10 sztuk).
- **Krok 3:** Klika "Kup".
- **Oczekiwany Rezultat:**
 - Wyświetla się potwierdzenie zlecenia.
 - Saldo gotówkowe użytkownika maleje o wartość transakcji + prowizję.
 - W portfelu pojawia się nowa pozycja (10 akcji AAPL).
 - W tabeli `portfolio.orders` pojawia się zlecenie ze statusem FILLED.

7.3.3 Scenariusz 3: Weryfikacja Alertów

- **Krok 1:** Użytkownik ustawia alert cenowy "Powyżej 150 USD".
- **Krok 2:** Symulacja zmiany ceny rynkowej powyżej progu.
- **Krok 3:** Uruchomienie skryptu `run_alert_evaluator.py`.
- **Oczekiwany Rezultat:** Skrypt wykrywa spełnienie warunku i aktualizuje status alertu w bazie danych.

7.4 Testy Automatyczne

7.4.1 Backend (Pytest)

Dla warstwy serwerowej przygotowano środowisko testowe oparte o framework **Pytest**. Testy obejmują:

- **Testy Jednostkowe:** Sprawdzanie poprawności funkcji obliczeniowych (np. wyliczanie PnL).
- **Testy Integracyjne API:** Wywoływanie endpointów za pomocą klienta testowego (TestClient z FastAPI) i weryfikacja kodów odpowiedzi HTTP oraz struktury zwracanego JSONa.

7.4.2 Frontend (Vitest)

Komponenty React są testowane przy użyciu narzędzia **Vitest** (kompatybilnego z Jest). Weryfikowane jest poprawne renderowanie komponentów oraz obsługa zdarzeń (np. kliknięcie przycisku).

7.5 Wnioski z testów

Przeprowadzone testy potwierdziły stabilność kluczowych funkcjonalności systemu. Wykryte błędy (np. problemy z formatowaniem liczb na wykresach) zostały na bieżąco poprawione.

Rozdział 8

Podsumowanie

8.1 Wnioski końcowe

Realizacja projektu FinBoard pozwoliła na stworzenie funkcjonalnej platformy do symulacji inwestycji giełdowych. Osiągnięto wszystkie główne cele założone na etapie planowania:

- Zbudowano nowoczesny frontend w technologii React, zapewniający wysoki komfort użytkowania.
- Stworzono wydajny backend w Python/FastAPI, obsługujący logikę finansową i autoryzację.
- Zaprojektowano i wdrożono rozbudowaną bazę danych PostgreSQL, przechowującą dane rynkowe i portfele użytkowników.
- Zapewniono możliwość łatwego wdrożenia systemu dzięki konteneryzacji Docker.

Projekt ten był doskonałą okazją do zgłębienia wiedzy na temat inżynierii oprogramowania w sektorze technologii finansowych (FinTech), a także do praktycznego zastosowania nowoczesnych wzorców architektonicznych.

8.2 Możliwości rozwoju

System FinBoard posiada otwartą architekturę, co pozwala na jego dalszą rozbudowę. Potencjalne kierunki rozwoju to:

1. **Moduł Analizy AI:** Integracja z modelami uczenia maszynowego do prognozowania trendów cenowych i sentymentu rynkowego (na bazie newsów).
2. **Integracja z rzeczywistymi brokerami:** Dodanie możliwości handlu na realnym rynku poprzez API domów maklerskich (np. Interactive Brokers).
3. **Aplikacja Mobilna:** Stworzenie natywnej aplikacji mobilnej (React Native) korzystającej z tego samego backendu.
4. **Social Trading:** Funkcje społecznościowe, pozwalające na udostępnianie swoich wyników i kopiowanie strategii innych użytkowników.

Projekt FinBoard stanowi solidny fundament pod budowę profesjonalnego narzędzia inwestycyjnego w przyszłości.

Bibliografia